

ARCHITECTURE SNAPSHOT

Myntra-Style Store Codebase Summary

A print-friendly walkthrough of the storefront demo, covering backend structure, frontend flow, integration points, execution paths, and onboarding cues for new contributors.

Express backend

React + Vite frontend

Generated 18 April 2026 at 8:22 pm

MYNTRA STYLE STORE

Document Guide

This PDF is styled for review and handoff: quick orientation at the top, then a section-by-section walkthrough of the codebase exactly as captured in the source Markdown file.

Reading tip Use the internal inventory and files-to-read-first sections to onboard quickly, then jump into the execution flow and risks.

Codebase Summary

Contents

Internal inventory

Backend entry points

Frontend entry points

Route definitions

API client definitions

Main state management files

Main config and env files

1. High-level project overview

Repository boundaries

Primary runtime story

2. Backend explanation

Folder-by-folder

Startup flow

Middleware

Routes, controllers, services,
models

Request lifecycle

Dependency flow

Files to read first

MYNTRA STYLE STORE

Risks

Codebase Summary

3. Frontend explanation

Folder-by-folder

Bootstrap flow

Routing

Pages, components, hooks, state,
API layer

Render and data flow

Component hierarchy

Files to read first

Risks

4. Frontend-backend flow

API definitions

Backend route mapping

Response handling

Concrete end-to-end journeys

5. Execution flow

6. Architecture summary

7. Onboarding guide

Fastest reading order

Productive first tasks

MYNTRA STYLE STORE

8. Text diagramCodebase Summary

Backend structure

Frontend structure

Data contract flow

9. Codebase risks and improvements

Evidence-based risks

Suggested improvements

Fact vs inference note

Internal inventory

Backend entry points

- `backend/index.js` is the backend entry point. It creates the Express app, defines the in-memory product catalog, exposes `GET /api/products`, and starts listening on `process.env.PORT || 5033`.

Frontend entry points

- `frontend/src/main.jsx` bootstraps React and renders `<App />` inside `StrictMode`.
- `frontend/src/App.jsx` contains the actual application shell, fetch logic, view composition, and a nested `ProductCard` component.

Route definitions

- `backend/index.js`
 - `GET /api/products`

API client definitions

- There is no dedicated API client module.
- `frontend/src/App.jsx` calls `fetch("http://localhost:5033/api/products")` directly inside a `useEffect`.

Main state management files

- `frontend/src/App.jsx`
 - `products`
 - `banner`
 - `status`
- There is no Redux, Context, Zustand, or custom hooks layer in the inspected source.

Main config and env files

- `backend/package.json` sets `index.js` as the main file and exposes `npm start`.
- `frontend/package.json` provides `dev`, `build`, `lint`, and `preview` scripts.
- `frontend/vite.config.js` uses the default React Vite plugin with no proxy or custom config.

- There is no `env` file in the inspected project, and the frontend hardcodes the backend URL.

MYNTRA STYLE STORE

Codebase Summary

1. High-level project overview

- **Fact** This repository is a compact full-stack storefront demo with an Express backend and a React + Vite frontend.
- **Fact** The backend serves a curated fashion-and-lifestyle catalog from an in-memory array declared directly in `backend/index.js`.
- **Fact** The frontend renders a branded landing page with navigation anchors, a hero section, category cards, product spotlight cards, and a footer with onboarding hints.
- **Inference** The project is designed as a polished training/demo app rather than a production e-commerce system, because the data layer, API surface, and frontend routing are intentionally minimal.

Repository boundaries

- `backend/`: Express API and sample data.
- `frontend/`: Vite + React single-page UI.
- `docs/`: not originally present in the repo; this summary is being added there as a documentation artifact.

Primary runtime story

- **Fact** The main user journey is a single page load.
- **Fact** On mount, `App.jsx` fetches catalog data from `GET /api/products`.
- **Fact** The backend formats prices using `Intl.NumberFormat("en-IN", ...)` before returning the response.
- **Fact** The frontend stores that response in state, derives categories with `useMemo`, and renders cards and status text from the fetched payload.

2. Backend explanation

Folder-by-folder

- `backend/package.json`
 - **Fact** The backend uses `express` and `cors`.
 - **Fact** The only script is `npm start`, which runs `node index.js`.

MYNTRA STYLE STORE

Codebase Summary

- **Fact** This file contains all backend logic: app setup, middleware, data, route handler, response shaping, and server startup.
- **Fact** There are no separate `routes/`, `controllers/`, `services/`, `models/`, or `data/` folders in this repo.

Startup flow

1. `npm start` runs `node index.js` from `backend/package.json`.
2. `backend/index.js` imports `express` and `cors`.
3. It creates the app and determines the port with `process.env.PORT || 5033`.
4. It enables `cors()` and `express.json()`.
5. It defines the in-memory `products` array.
6. It registers `GET /api/products`.
7. It starts listening and logs `Catalog API listening on http://localhost:${PORT}`.

Middleware

- **Fact** `cors()` is globally enabled.
- **Fact** `express.json()` is globally enabled.
- **Fact** There is no custom error middleware and no 404 fallback route.
- **Inference** `express.json()` is defensive scaffolding for future write endpoints, because the current API only serves a read-only `GET`.

Routes, controllers, services, models

Routes

- `GET /api/products`
 - Defined inline in `backend/index.js`.
 - No route module abstraction exists.

Controllers

- **Fact** There is no separate controller layer.
- **Inference** The anonymous route handler in `app.get("/api/products", ...)` is effectively acting as both controller and service.

Services

MYNTRA STYLE STORE

Codebase Summary

- **Fact** There is no separate service module.
- **Fact** The route handler maps over `products` and augments each item with `priceFormatted`.
- **Fact** It uses `new Intl.NumberFormat("en-IN", { style: "currency", currency: product.currency })` to produce locale-aware pricing strings.

Models / schemas

- **Fact** There are no schemas or ORM models.
- **Fact** The product shape is implied by plain JavaScript objects in the `products` array.
- **Fact** Each product currently includes `id`, `name`, `price`, `currency`, `badge`, `category`, `description`, and `delivery`.

Request lifecycle

1. The browser requests `http://localhost:5033/api/products`.
2. Express matches the inline `app.get("/api/products", ...)` handler.
3. The handler maps the in-memory `products` array into a `curated` array.
4. Each product is copied and extended with `priceFormatted`.
5. The handler returns JSON in the shape `{ banner, curated }`.
6. The browser receives the payload and the frontend updates state.

Dependency flow

- `backend/index.js`
 - depends on `express`
 - depends on `cors`
 - contains the data source
 - contains the transformation logic
 - contains the route definition
- **Inference** This is the simplest possible vertical slice, which makes onboarding fast but leaves little separation for future expansion.

Files to read first

- `backend/index.js`: the single most important backend file because it contains the entire API surface and data contract.

- `backend/package.json` : confirms the backend runtime and dependency versions.

MYNTRA STYLE STORE

Codebase Summary

Risks

- **Fact** All data is in-memory, so there is no persistence or external data source.
- **Fact** There is only one API endpoint, which means the app has no product detail, filtering, search, or cart APIs.
- **Fact** There is no validation, logging middleware, error middleware, or test coverage in the inspected source.
- **Fact** The response banner contains mojibake characters: `Festival Drop ï¿½ Free COD ï¿½ Express delivery`.
- **Inference** If the catalog grows, `index.js` will become harder to maintain because data, formatting, routing, and startup are all coupled together.

3. Frontend explanation

Folder-by-folder

- `frontend/package.json`
 - **Fact** Uses React 19, React DOM 19, Vite 8, and ESLint 9.
- `frontend/vite.config.js`
 - **Fact** Uses `@vitejs/plugin-react` with default configuration.
 - **Fact** No development proxy is configured.
- `frontend/src/main.jsx`
 - **Fact** Bootstraps the React app.
- `frontend/src/App.jsx`
 - **Fact** Holds almost all application behavior and markup.
 - **Fact** Declares `navLinks`, `architecturePillars`, `ProductCard`, and `App` in the same file.
- `frontend/src/App.css`
 - **Fact** Contains most component-level styling for layout sections, cards, hero, footer, and responsive behavior.
- `frontend/src/index.css`

◦ **Fact** Sets theme variables, imports fonts from Google Fonts, and establishes base styles.

MYNTRA STYLE STORE Codebase Summary

- `frontend/public/`
 - **Fact** Contains shared static assets such as `icons.svg` and `favicon.svg`.
- `frontend/src/assets/`
 - **Fact** Contains bundled assets including `hero.png`, `react.svg`, and `vite.svg`.
 - **Fact** `hero.png` is present in the repo but is not imported or referenced from `App.jsx`.

Bootstrap flow

1. Vite serves `frontend/index.html`.
2. `frontend/src/main.jsx` mounts the React tree.
3. `App.jsx` initializes `products`, `banner`, and `status`.
4. The initial `useEffect` calls the backend endpoint exactly once on mount.
5. If the response is successful, the app stores `banner` and `curated` products and marks `status` as `ready`.
6. If the request fails, the app logs the error and marks `status` as `error`.
7. `useMemo` derives up to four distinct categories from the fetched products.

Routing

- **Fact** There is no client-side router.
- **Fact** Navigation is in-page anchor navigation using `href="#collections"`, `href="#architecture"`, `href="#stories"`, and `href="#docs"`.
- **Inference** The anchors are there to simulate section-based navigation in a landing page rather than true page-level navigation.

Pages, components, hooks, state, API layer

Pages / components

- `App`
 - **Fact** Acts as the only page component.
 - **Fact** Renders the header, hero, category panel, product spotlight section, architecture section, and footer.
- `ProductCard`

Fact Nested inside `App.jsx`.

MYNTRA STYLE STORE

Codebase Summary

- Fact Receives a `product` prop and displays `badge`, `priceFormatted`, `description`, `category`, and `delivery`.
- Fact Uses a decorative empty `div.product-image` instead of a real product image.

Hooks

- `useState`
 - Fact Tracks `products`, `banner`, and `status`.
- `useEffect`
 - Fact Triggers the initial catalog fetch.
- `useMemo`
 - Fact Derives the `categories` array from unique product categories.

State

- `products`
 - Fact Drives the product grid and category derivation.
- `banner`
 - Fact Drives the first hero highlight chip.
- `status`
 - Fact Controls hero status text, the spotlight status pill, and the error/loading states.

API layer

- Fact There is no shared client, hook, or service abstraction.
- Fact `fetch("http://localhost:5033/api/products")` is hardcoded in `App.jsx`.
- Fact Because there is no Vite proxy and no env-driven base URL, frontend and backend coordination currently depends on that fixed port.

Render and data flow

- Header renders immediately from static `navLinks`.
- Hero renders with static marketing copy and dynamic `banner` / `status`.
- When the fetch succeeds, `products` is populated from `data.curated`.
- `categories` is recomputed from the current product list.

5. The category panel renders live categories if data exists, otherwise a fallback array of **MYNTRA STYLE STORE** Codebase Summary
["Ethnic", "Western", "Gadgets", "Beauty"] .

6. The spotlight section renders loading text, an error message, or mapped **ProductCard** instances depending on **status** .
7. The architecture section renders static cards from **architecturePillars** .
8. The footer renders static onboarding notes and run instructions.

Component hierarchy

```
main.jsx
-> App
  -> header.main-header
    -> logo
    -> nav links
    -> CTA button
  -> main
    -> section.hero
      -> hero-content
      -> hero-panel
    -> section.category-panel
      -> category cards
    -> section.products-section
      -> products-header
      -> status text (conditional)
      -> product-grid
        -> ProductCard x N
    -> section.tech-section
      -> architecture pillar cards
  -> footer.main-footer
```

Files to read first

- **frontend/src/App.jsx** : contains the live runtime behavior and is the highest-value frontend file.
- **frontend/src/App.css** : maps each rendered section to its layout and visual treatment.
- **frontend/src/main.jsx** : confirms the actual entry point.
- **frontend/vite.config.js** : useful to verify there is no dev proxy or special environment setup.
- **frontend/package.json** : confirms scripts and framework versions.

Risks

MYNTRA STYLE STORE

Codebase Summary

- **Fact** `App.jsx` combines page composition, data fetching, derived state, copywriting, and component definitions in one file.
- **Fact** The frontend hardcodes `http://localhost:5033/api/products` instead of using environment configuration.
- **Fact** `frontend/src/index.css` imports Google Fonts over the network, which can affect rendering in offline or restricted environments.
- **Fact** Multiple strings contain encoding artifacts such as `Login • Explore`, `Inspired by Myntra • Built for India`, and `Loading productsâ€¦`.
- **Fact** `hero.png` exists in `src/assets` but is unused, which suggests dead or incomplete design work.

4. Frontend-backend flow

API definitions

- `GET /api/products`
 - Response shape:
 - `banner: string`
 - `curated: Product[]`
 - Each `Product` includes the original fields plus `priceFormatted`.

Backend route mapping

- `frontend/src/App.jsx`
 - `useEffect(fetchCatalog)` calls `GET /api/products`
- `backend/index.js`
 - `app.get("/api/products", ...)` handles the request
 - response is shaped inline and returned with `res.json(...)`

Response handling

- **Fact** The frontend expects `data.banner` and `data.curated`.
- **Fact** `setBanner(data.banner)` feeds the hero highlight.

- **MYNTRA STYLE STORE** `setProducts(data.curated)` feeds both the category derivation and the product grid. Codebase Summary

- **Fact** `setStatus("ready")` switches the page from loading state to ready state.

Concrete end-to-end journeys

Journey 1: Initial storefront load

1. User opens the frontend.
2. `App` mounts and runs `fetchCatalog()`.
3. The browser requests `GET /api/products`.
4. The backend enriches each product with `priceFormatted`.
5. The frontend stores `banner` and `products`.
6. The hero, category cards, and product spotlight all re-render with live data.

Journey 2: Successful category rendering

1. The product response arrives with categories such as `Ethnic`, `Western`, `Gadgets`, and `Accessories`.
2. `useMemo` builds a `Set` from `products.map((product) => product.category)`.
3. The resulting unique categories are sliced to the first four values.
4. The category panel renders those categories instead of the fallback hardcoded set.

Journey 3: Backend unavailable

1. The frontend fetch fails or returns a non-OK status.
2. `fetchCatalog()` throws or enters the `catch`.
3. The app logs the error to the console.
4. `status` becomes `error`.
5. The hero panel switches to `Catalog unreachable`, and the product section prompts the user to start the backend with `npm start`.

5. Execution flow

```
Frontend boot
index.html
-> src/main.jsx
-> App.jsx
```

```
-> useEffect(fetchCatalog)
```

MYNTRA STYLE STORE

Codebase Summary

```
-> GET http://localhost:5033/api/products
```

```
-> backend/index.js
```

```
-> map products
```

```
-> add priceFormatted
```

```
-> return { banner, curated }
```

```
-> setBanner
```

```
-> setProducts
```

```
-> setStatus("ready")
```

```
-> render hero, categories, cards, architecture, footer
```

6. Architecture summary

- **Fact** The backend is intentionally flat: one file, one dataset, one route.
- **Fact** The frontend is also mostly flat, with one page component and one nested presentational component.
- **Fact** The integration contract is tiny and easy to understand because the frontend only depends on one JSON response shape.
- **Inference** This repo is optimized for quick onboarding and UI iteration, not for multi-team ownership or long-term feature growth.
- **Inference** The first scalability refactor would likely be splitting the backend into route/service/data files and moving frontend fetch logic into a reusable API module.

7. Onboarding guide

Fastest reading order

1. Read `backend/index.js` to understand the exact API contract and real backend port.
2. Read `frontend/src/App.jsx` to see how the page consumes that contract.
3. Read `frontend/src/App.css` to map the major UI sections to their visual intent.
4. Read `frontend/src/index.css` and `frontend/vite.config.js` to understand global styling and build setup.
5. Read `frontend/package.json` and `backend/package.json` to confirm framework/runtime versions.

Productive first tasks

MYNTRA STYLE STORE

Codebase Summary

- Extract the backend `products` data into its own module or JSON file.
- Move the inline fetch call into a small API helper and read the base URL from `import.meta.env`.
- Split `App.jsx` into section components such as `Header`, `Hero`, `CategoryPanel`, `ProductsSection`, and `ArchitectureSection`.
- Add a second endpoint for product details or category filtering to make the app less hardcoded.
- Fix the mojibake strings so the UI copy renders correctly.

8. Text diagrams

Backend structure

```
backend/index.js
-> app setup
-> middleware
-> in-memory products[]
-> GET /api/products
-> response shaping (priceFormatted)
-> app.listen()
```

Frontend structure

```
src/main.jsx
-> App.jsx
  -> ProductCard
  -> fetch catalog
  -> derive categories
  -> render sections
```

Data contract flow

```
products[] in backend
-> enrich with Intl.NumberFormat
-> { banner, curated }
```



```
-> frontend setBanner + setProducts  
MYNTRA STYLE STORE  
-> hero highlights
```

Codebase Summary

```
-> category cards  
-> product spotlight grid
```

9. Codebase risks and improvements

Evidence-based risks

- **Fact** There is no `docs/` folder in the original repo even though the UI footer points engineers toward documentation and mentor notes.
- **Fact** The frontend hardcodes backend origin and port, creating local coupling.
- **Fact** There is no proxy config in `frontend/vite.config.js`.
- **Fact** The UI claims architectural ideas like telemetry and observability, but no such code exists in the current source.
- **Fact** The backend does not expose health, detail, or category-specific endpoints despite the UI's broader architectural framing.
- **Fact** There are visible encoding issues in both frontend and backend strings.

Suggested improvements

- Add `docs/` permanently with onboarding notes and API contracts.
- Introduce `VITE_API_URL` or a Vite dev proxy.
- Add a `GET /api/products/:id` endpoint and link cards to a real detail view.
- Extract data and formatting helpers from `backend/index.js`.
- Add request/error states that are more user-visible than console logging.
- Either use `hero.png` in the hero section or remove it to keep the asset folder tidy.

Fact vs inference note

- Fact labels above are grounded in inspected files under `backend/` and `frontend/`.
- Inference labels mark likely intent, design direction, or future-friendly interpretations that are not explicitly implemented in code.